

DOCUMENTO D7 — SPECDRIVE AURA

Organizacao de Agentes

Estrutura de 3 agentes por funcao:
Orquestrador, Executor, Corretor

1. Filosofia dos 3 Agentes

Para garantir qualidade, consistencia e velocidade no desenvolvimento da Aura, cada funcao do projeto tera **3 agentes de IA** trabalhando em conjunto:

1. Orquestrador (O) — "O Arquiteto"

Funcao: Planeja, decide e coordena. E o agente estrategico que define O QUE fazer, POR QUE fazer, e EM QUE ORDEM.

- Analisa requisitos e decide a melhor abordagem
- Divide tarefas em subtarefas menores
- Define prioridades e dependencias
- Cria especificacoes tecnicas para os executores
- Toma decisoes arquiteturais

2. Executor (E) — "O Construtor"

Funcao: Implementa, codifica e constroi. E o agente operacional que executa as tarefas definidas pelo Orquestrador.

- Escreve codigo (React, TypeScript, SQL, Edge Functions)
- Cria componentes, paginas, APIs
- Implementa designs e layouts
- Integra com servicos externos (Stripe, Woovi)
- Realiza deploys

3. Corretor (C) — "O Revisor"

Funcao: Revisa, testa e corrige. E o agente de qualidade que verifica o trabalho do Executor e garante que esta correto.

- Revisa codigo (code review)
- Identifica bugs, falhas de seguranca, problemas de performance
- Verifica se o codigo segue os padroes definidos
- Testa funcionalidades (manual e automatizado)
- Sugere melhorias e otimizacoes
- Valida que o resultado atende a especificacao do Orquestrador

2. Estrutura por Funcao

2.1 Funcao: Backend/Supabase

Tabela 1 Agentes — Backend

Agente	Nome	Responsabilidades	Entregaveis
Orquestrador	Backend-O	Definir schema do banco, RLS policies, indexes, triggers. Decidir arquitetura de dados.	Schema SQL completo, diagrama ER, plano de migrations
Executor	Backend-E	Criar tabelas, implementar RLS, triggers, functions. Escrever Edge Functions.	Codigo SQL, migrations, Edge Functions deployadas
Corretor	Backend-C	Revisar schema, testar RLS (tentar burlar), verificar performance de queries, validar seguranca.	Relatorio de review, testes de penetracao, benchmarks

2.2 Funcao: Frontend/Integracao

Tabela 2 Agentes — Frontend

Agente	Nome	Responsabilidades	Entregaveis
Orquestrador	Frontend-O	Decidir quais paginas conectar primeiro, definir hooks/padrees, planejar estado global (Zustand).	Plano de integracao, arquitetura de estado, definicao de hooks
Executor	Frontend-E	Integrar Supabase Client, criar hooks useQuery, conectar paginas as APIs, implementar Zustand stores.	Codigo React integrado, hooks customizados, stores Zustand
Corretor	Frontend-C	Revisar integracao, testar fluxos completos (login → compra → check-in), verificar loading states e erros.	Relatorio de testes, lista de bugs, sugestoes de UX

2.3 Funcao: Pagamentos (Stripe + Woovi)

Tabela 3 Agentes — Pagamentos

Agente	Nome	Responsabilidades	Entregaveis
Orquestrador	Payments-O	Definir fluxo de pagamento completo, split de receita, conciliacao, webhooks.	Diagrama de fluxo de pagamento, spec de split, plano de testes
Executor	Payments-E	Implementar Stripe Connect, Woovi integration, webhooks, tabela de payments, geracao de QR Code.	Edge Functions, componentes de checkout, confirmacao de pagamento
Corretor	Payments-C	Testar fluxo completo de pagamento (Pix + cartao), verificar split, testar webhooks, testar reembolso.	Relatorio de testes de pagamento, validacao de conciliacao

2.4 Funcao: Auth e Seguranca

Tabela 4 Agentes — Auth

Agente	Nome	Responsabilidades	Entregaveis
Orquestrador	Auth-O	Definir fluxo de auth, roles, permissoes, OAuth providers, recuperacao de senha.	Spec de auth, matriz de permissoes, fluxo de onboarding
Executor	Auth-E	Implementar Supabase Auth, AuthContext real, Google OAuth, ProtectedRoute, perfil de usuario.	Sistema de auth funcional, login, registro, logout, perfil
Corretor	Auth-C	Testar seguranca: tentar acesso nao autorizado, testar RLS, testar SQL injection, testar XSS.	Relatorio de seguranca, testes de penetracao

2.5 Funcao: DevOps/Infra

Tabela 5 Agentes — DevOps

Agente	Nome	Responsabilidades	Entregaveis
Orquestrador	DevOps-O	Definir pipeline CI/CD, ambientes, variaveis de ambiente, backup, monitoramento.	Plano de infra, pipeline CI/CD, matriz de ambientes
Executor	DevOps-E	Configurar Vercel, Supabase, GitHub Actions, DNS, dominio, SSL, Storage.	Infra configurada, deploy automatico, ambiente produtivo
Corretor	DevOps-C	Verificar configuracoes, testar deploy, testar rollback, verificar seguranca de configs.	Auditoria de infra, testes de disaster recovery

3. Workflow de Colaboracao

3.1 Ciclo de Trabalho (Sprint de 1 Semana)

Dia 1 – Orquestrador (O) planeja:

- Definir o que sera feito na semana
- Criar especificacoes tecnicas
- Dividir em tarefas pequenas

Dia 1-3 – Executor (E) implementa:

- Escrever codigo segundo as especificacoes
- Fazer deploy em staging
- Documentar o que foi feito

Dia 3-4 – Corretor (C) revisa:

- Code review completo
- Testes de funcionalidade
- Reportar bugs e sugestoes

Dia 4-5 – Executor (E) corrige:

- Aplicar feedback do Corretor
- Corrigir bugs encontrados
- Deploy final em producao

Fim da semana:

- Corretor valida as correcoes
- Orquestrador planeja proxima semana

3.2 Comunicacao entre Agentes

Protocolo de Comunicacao

1. **O → E**: Orquestrador entrega especificacao clara com criterios de aceitacao
2. **E → C**: Executor entrega codigo com descricao do que foi feito e como testar
3. **C → E**: Corretor entrega review com: aprovado / aprovado com ressalvas / reprovado
4. **C → O**: Corretor reporta problemas estruturais ao Orquestrador
5. **O → C**: Orquestrador esclarece requisitos quando o Corretor identifica ambiguidade

3.3 Definicao de "Pronto" (Definition of Done)

Tabela 6 Checklist — Definition of Done

Criterio	Responsavel
Codigo escrito e funcionando	Executor
Testes manuais passaram	Executor
Code review aprovado pelo Corretor	Corretor
Sem bugs criticos ou de seguranca	Corretor
RLS policias testadas e funcionando	Corretor

Deploy em staging funcionando

Executor

Documentacao atualizada (se necessario)

Executor

Aprovado pelo Orquestrador

Orquestrador

4. Comandos Militares — Instruções para Agentes

Instruções Universais (aplicam a todos os agentes)

1. **SEMPRE verificar o todo antes de mudar uma parte.** Leia o contexto completo antes de modificar qualquer arquivo.
2. **SEMPRE testar antes de entregar.** O código deve funcionar. Não entregue código quebrado.
3. **SEMPRE seguir os padrões existentes.** Use a mesma estrutura, nomenclatura e estilo do código já existente.
4. **SEMPRE documentar decisões importantes.** Se você tomou uma decisão arquitetural, explique por que.
5. **NUNCA inventar dados ou estatísticas.** Se precisar de números, busque ou diga que não tem.
6. **NUNCA deletar código sem verificar dependências.** Verifique quem usa aquele componente/função antes de remover.
7. **NUNCA commitar código com dados sensíveis.** Chaves API, senhas, tokens NUNCA no código.
8. **NUNCA deixar console.log em produção.** Remova todos os logs de debug antes do deploy.
9. **SEMPRE usar TypeScript strict.** Não use "any". Tipagem forte em todo código novo.
10. **SEMPRE pensar em segurança.** Sanitize inputs, valide dados, proteja contra XSS e SQL injection.

Instruções para Orquestradores

1. **SEJA CONSERVADOR em estimativas.** É melhor sub-prometer e super-entregar.
2. **DIVIDA em partes pequenas.** Tarefas de no máximo 2-3 dias de trabalho.
3. **DEFINA critérios de aceitação claros.** "Funcionar" não é suficiente — defina o que "funcionar" significa.
4. **CONSIDERE o custo de manutenção.** Código complexo é mais caro no longo prazo.
5. **PRIORIZE o que gera valor.** Features que fazem dinheiro vêm antes de features "legais".

Instruções para Executores

1. **ESCREVA código limpo e legível.** Nomes claros, funções pequenas, sem duplicação.
2. **USE componentes shadcn/ui** sempre que possível. Não reinvente a roda.
3. **IMPLEMENTE loading states e error states** em TODAS as chamadas a API.
4. **TESTE manualmente** cada fluxo antes de marcar como pronto.
5. **COMMIT frequentemente** com mensagens descritivas (conventional commits).

Instruções para Corretores

1. **SEJA RIGOROSO mas construtivo.** Aponte o problema, sugira a solução.
2. **TESTE como um usuário real.** Não só leia o código — use a aplicação.
3. **VERIFIQUE segurança em primeiro lugar.** Bugs de segurança são P0 sempre.
4. **DOCUMENTE tudo que encontrar.** Bugs, sugestões, elogios — tudo por escrito.
5. **APRENDA com os erros.** Se um tipo de bug repete, sugira um padrão para prevenir.

5. Estrutura de Skills e Conhecimento

5.1 Skills que Todo Agente Deve Ter

Tabela 7 Skills Obrigatorias

Skill	Nivel	Obrigatorio
React 19 + TypeScript	Avancado	Sim (Frontend)
Tailwind CSS + shadcn/ui	Avancado	Sim (Frontend)
Supabase (Postgres, Auth, Storage)	Avancado	Sim (Backend)
Stripe Connect API	Intermediario	Sim (Payments)
SQL (PostgreSQL)	Avancado	Sim (Backend)
Deno / TypeScript (Edge Functions)	Intermediario	Sim (Backend)
Seguranca web (OWASP Top 10)	Intermediario	Sim (Todos)
Git + GitHub Actions	Intermediario	Sim (Todos)

5.2 Documentacao de Referencia

Tabela 8 Documentos de Referencia para Agentes

Documento	Conteudo	Quando Usar
D1 — Inventario Frontend	Tudo que ja existe no frontend: 66 paginas, 82 componentes, stack tecnologico	Antes de modificar qualquer pagina ou componente
D2 — Gap Analysis	Tudo que falta fazer: tabelas, APIs, integracoes, por prioridade	Para planejar o que fazer na proxima sprint
D3 — Analise Estrategica	Decisoes estrategicas: precos, stack, roadmap, KPIs	Para tomar decisoes arquiteturais
D4 — Frontend Melhorias	O que melhorar no frontend: UX, performance, a11y, SEO, novas features	Antes de implementar qualquer melhoria no frontend
D5 — Supabase Spec	Schema SQL completo, RLS, triggers, Edge Functions, Storage, Auth config	Para criar ou modificar qualquer tabela, policy ou function
D6 — Arquitetura Sistemas	Todos os sistemas: Supabase, Vercel, Stripe, Woovi, Resend, etc.	Para configurar ou integrar qualquer servico externo
D7 — Organizacao Agentes	Estrutura de 3 agentes, workflow, definicao de pronto, comandos militares	Diariamente — para lembrar como trabalhar

Regra de Ouro

NENHUM agente deve escrever uma linha de codigo sem antes ler o documento relevante (D1-D6). A qualidade do output e diretamente proporcional ao conhecimento do contexto. Estes 7 documentos sao o "cerebro coletivo" da Aura.

